Proceso de implementación a partir de un diseño.

Semana 6

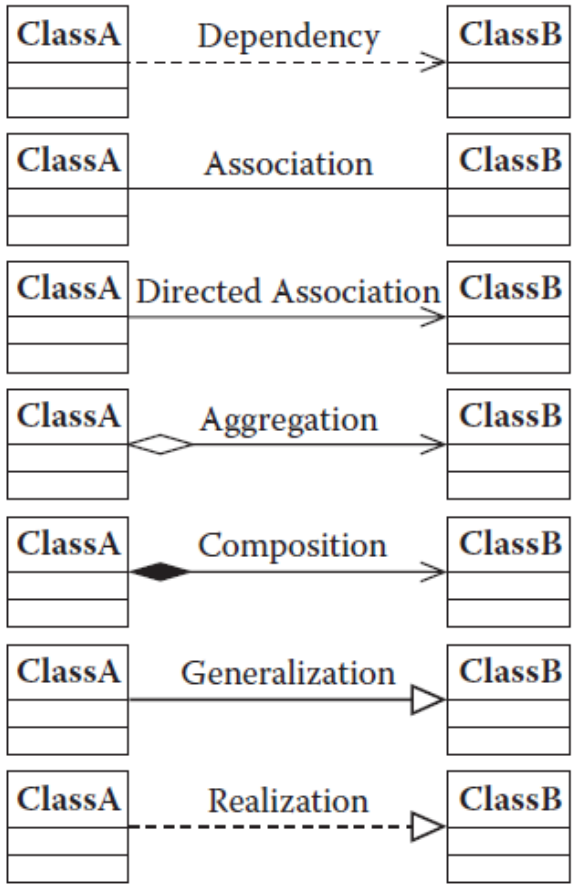
Agenda

- Traduciendo un diagrama de clase a código fuente y viceversa
- Traduciendo un diagrama de secuencia a código fuente y viceversa

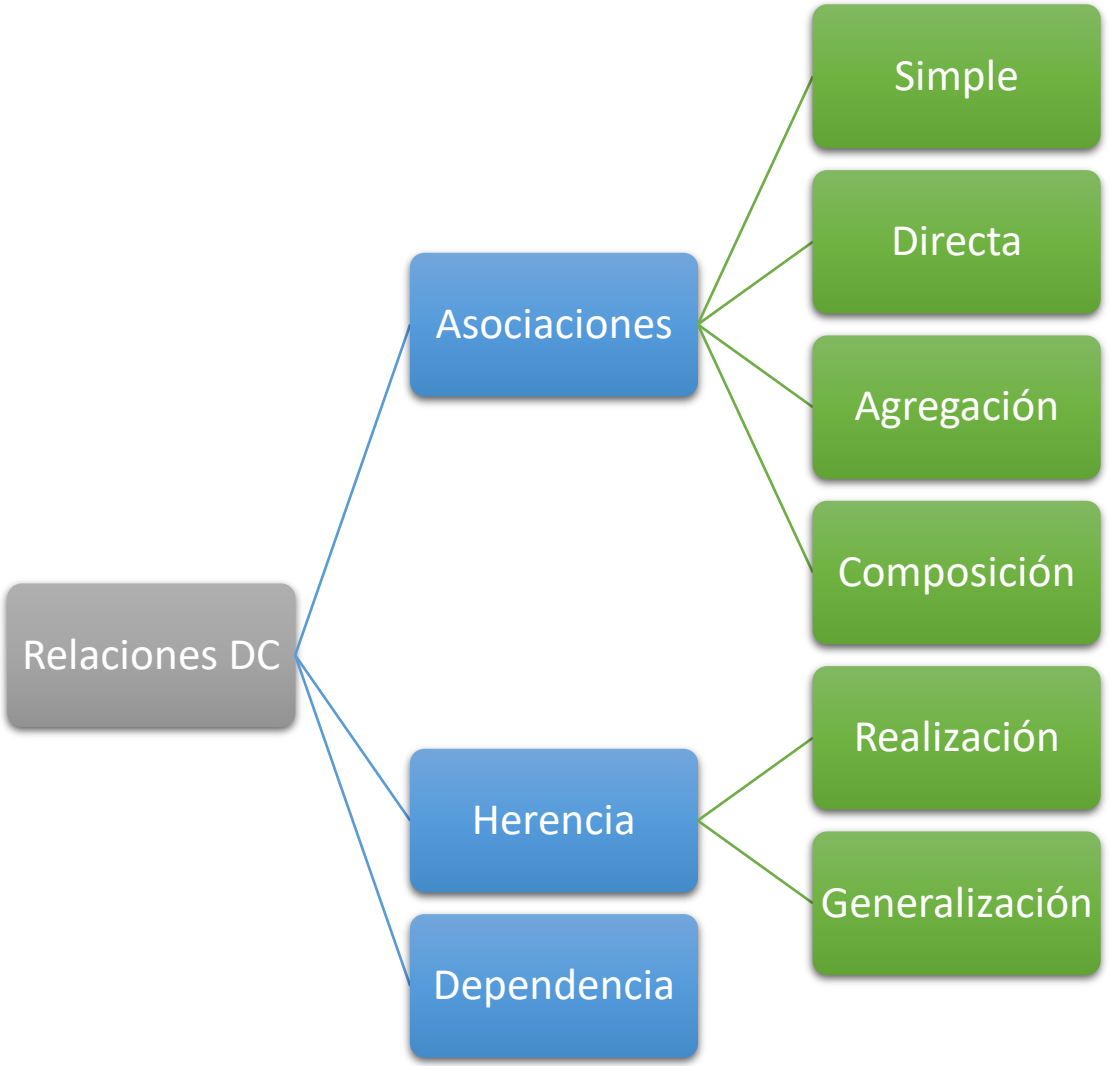
Traduciendo un diagrama de clase a código fuente y viceversa

Relaciones en Diagramas de Clases

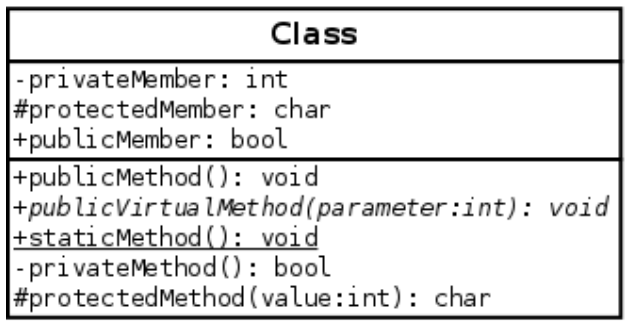
- Las clases deben estar relacionadas con otras.



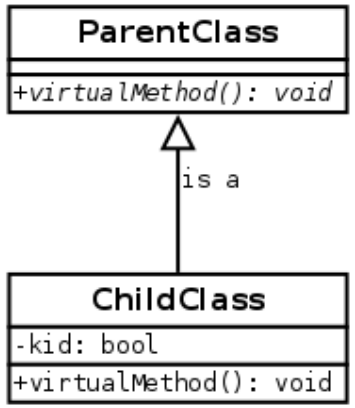
Relaciones en DC



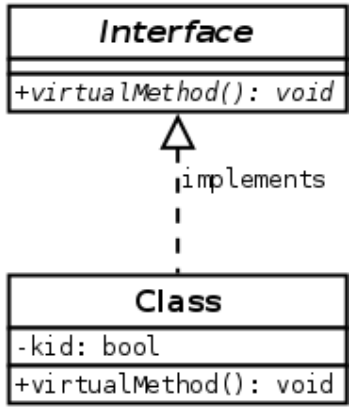
UML Class Diagram Cheat Sheet



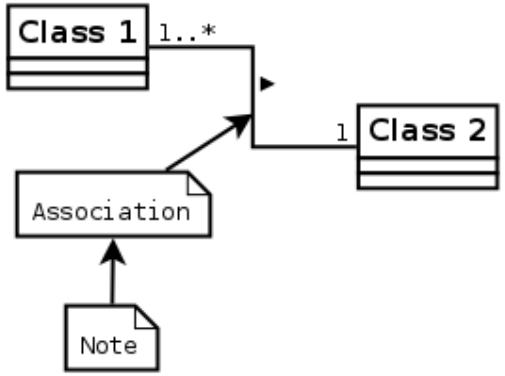
Generalization



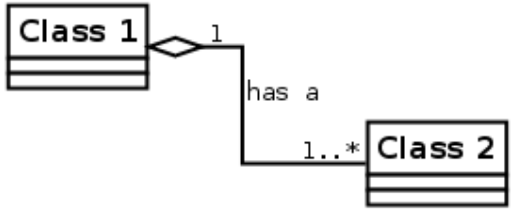
Realization



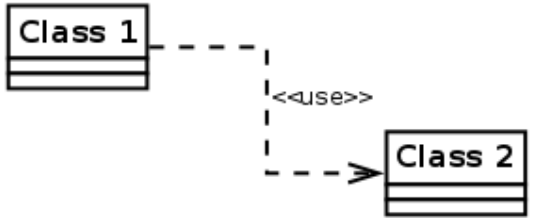
Association



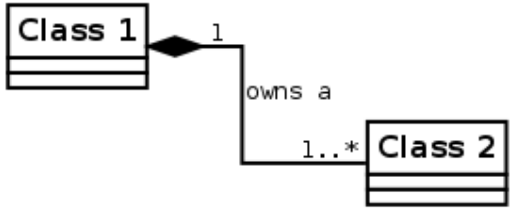
Aggregation



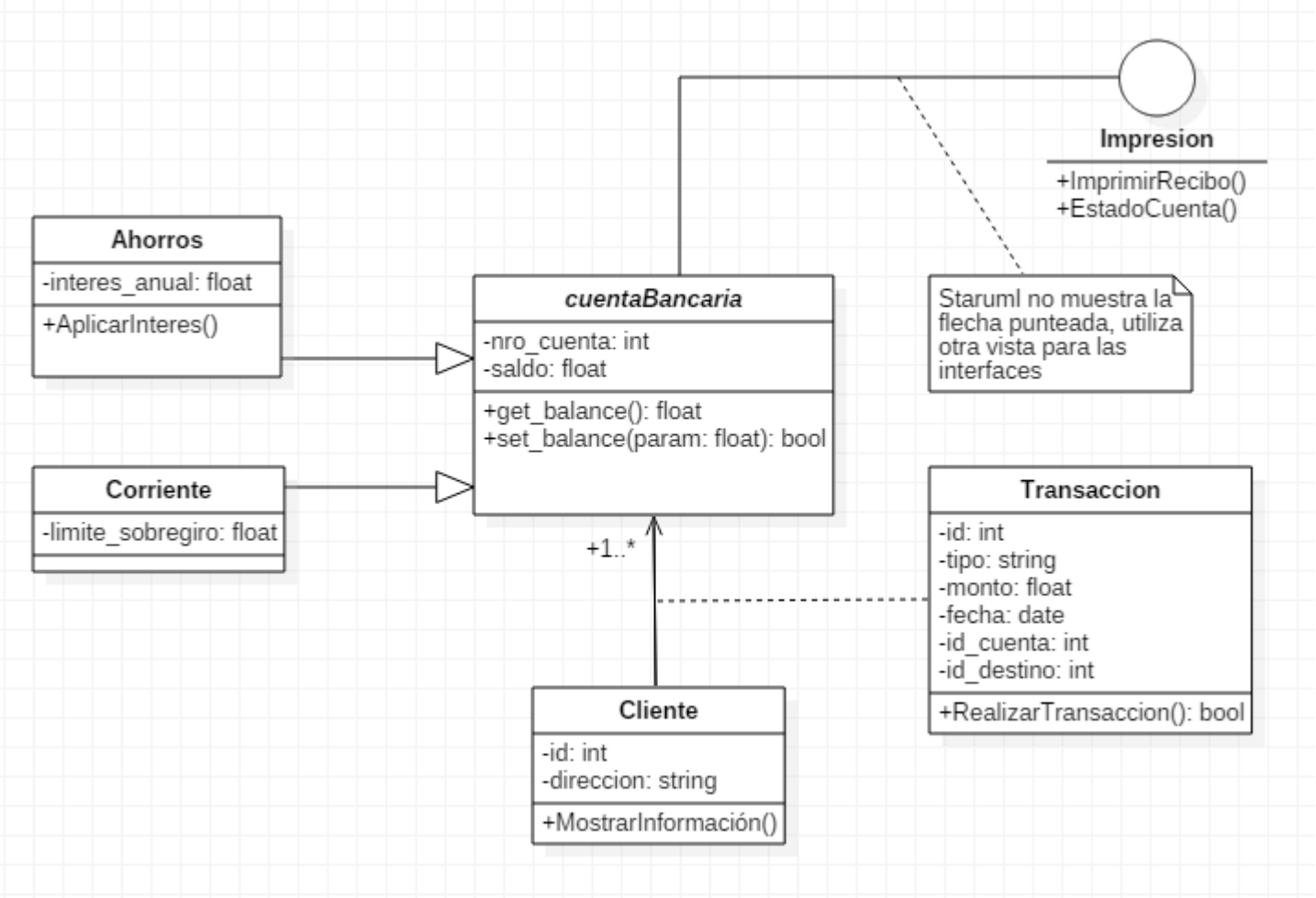
Dependency



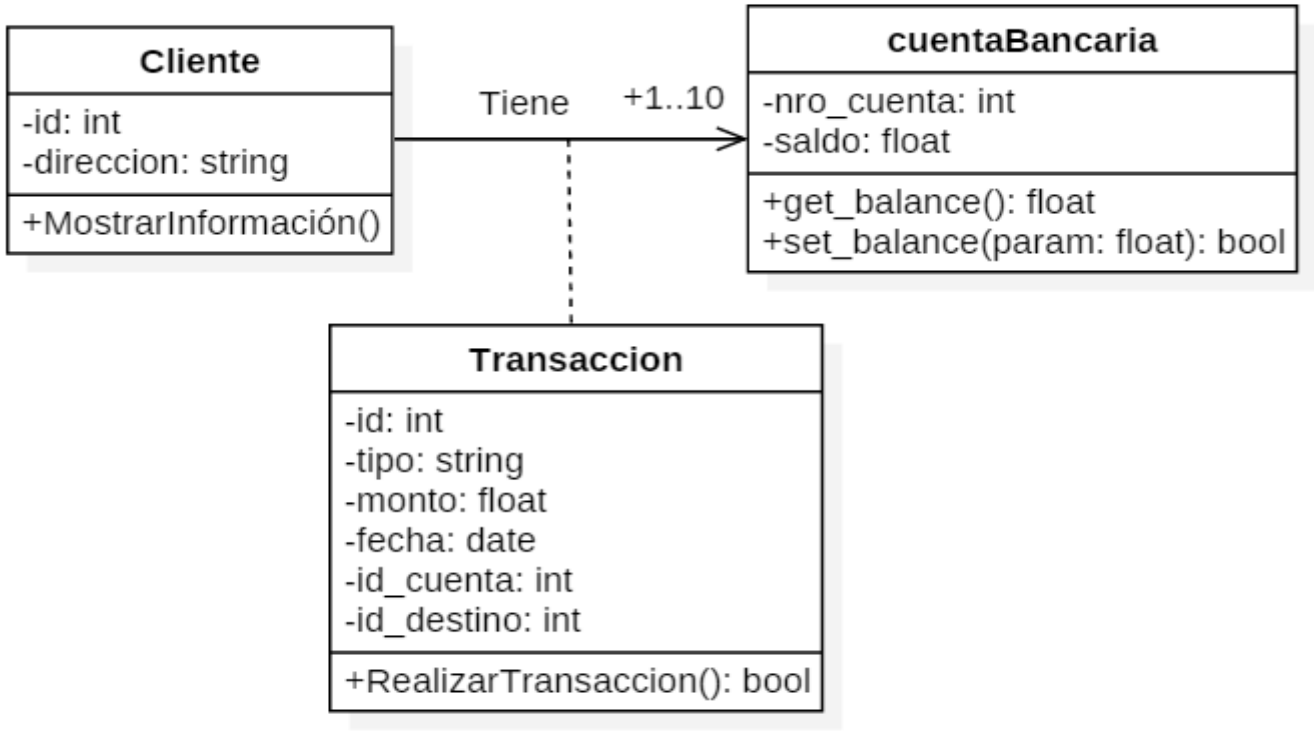
Composition



Ejemplo de Diagrama de clases



Asociación


Cliente
-id: int
-direccion: string
+MostrarInformación()
Tiene
+1..10
cuentaBancaria
-nro_cuenta: int
-saldo: float
+get_balance(): float
+set_balance(param: float): bool
Transaccion
-id: int
-tipo: string
-monto: float
-fecha: date
-id_cuenta: int
-id_destino: int
+RealizarTransaccion(): bool

Código - Asociación

```java

import java.util.*;

/**
 *
 */
public abstract class CuentaBancaria implements Impresion {

    /**
     * Default constructor
     */
    public CuentaBancaria() {
    }

    private int nro_cuenta;

    private float saldo;


    public float get_balance() {
        // TODO implement here
        return 0.0f;
    }

    public boolean set_balance(float param) {
        // TODO implement here
        return null;
    }

    public void ImprimirRecibo() {
        // TODO implement here
    }

    public void EstadoCuenta() {
        // TODO implement here
    }


}
```

```java

import java.util.*;

/**
 *
 */
public class Cliente {

    /**
     * Default constructor
     */
    public Cliente() {
    }

    private int id;

    private String direccion;

    public ArrayList<CuentaBancaria> cuentas;

    public void MostrarInformación() {
        // TODO implement here
    }


}
```

Clase de asociación

- La clase de asociación se instancia al ejecutar la relación entre Cliente y CuentaBancaria.
- RealizarTransaccion () asocia los clientes y las cuentas.

```java

import java.util.*;

/**
 *
 */
public class Transaccion {

    /**
     * Default constructor
     */
    public Transaccion() {
    }

    private int id;

    private String tipo;

    private float monto;

    private Date fecha;

    private int id_cuenta;

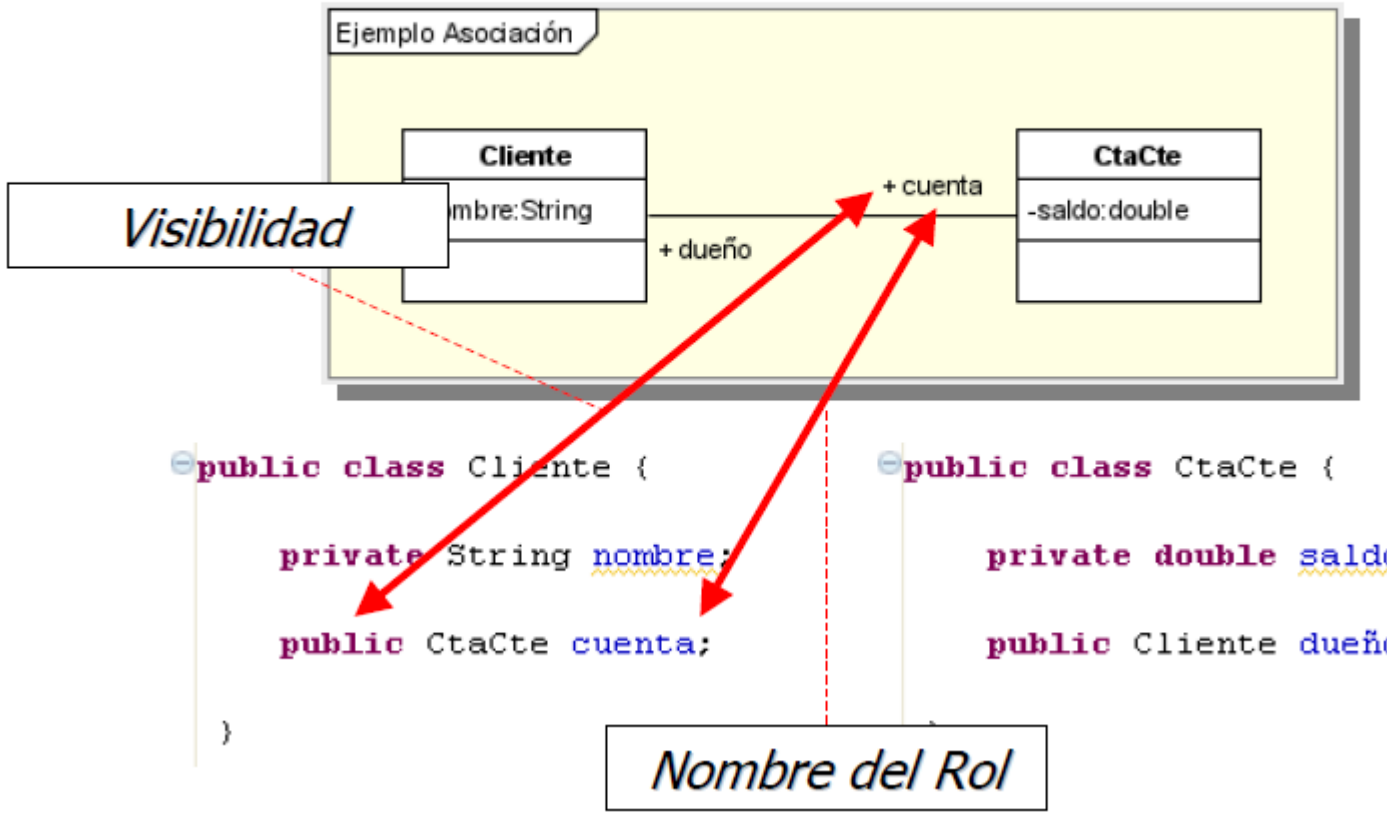
    private int id_destino;

    /**
     * @return
     */
    public boolean RealizarTransaccion(int id_cuenta, int id_destino) {
        // TODO implement here
        return null;
    }

}
```

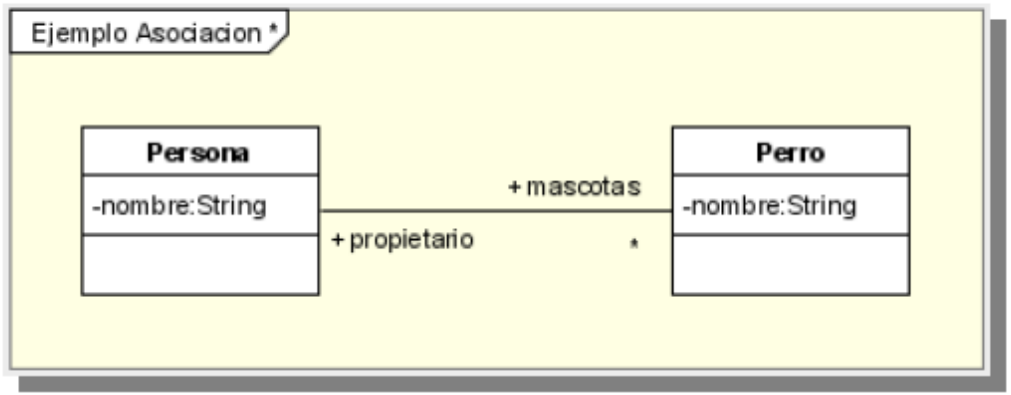
Bidireccionalidad

- Bidireccional con multiplicidad 0..1 o 1



Bidireccionalidad

- Bidireccional con multiplicidad *




```java
public class Persona {

    private String nombre;

    public java.util.Collection mascotas = new java.util.TreeSet();

}
```

```java
public class Perro {

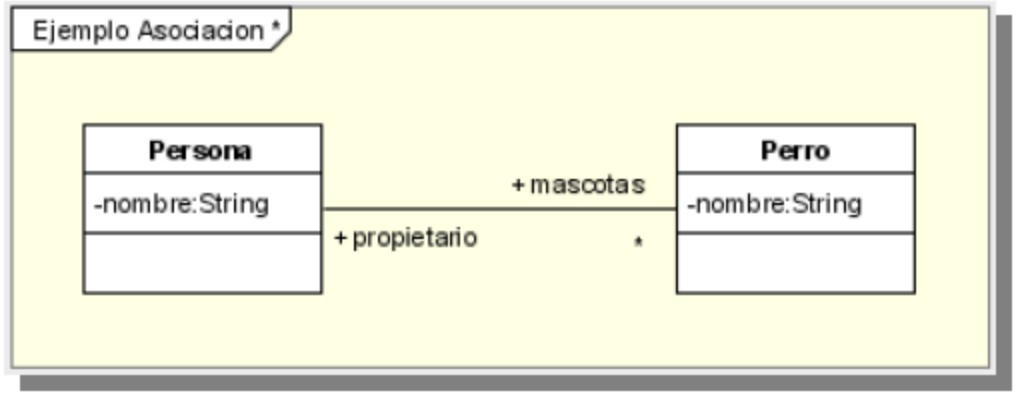
    private String nombre;

    public Persona propietario;

}
```

Bidireccionalidad

- Bidireccional con multiplicidad *




```java
public class Persona {

    private String nombre;

    public java.util.Collection mascotas = new java.util.TreeSet();

}
```

```java
public class Perro {

    private String nombre;

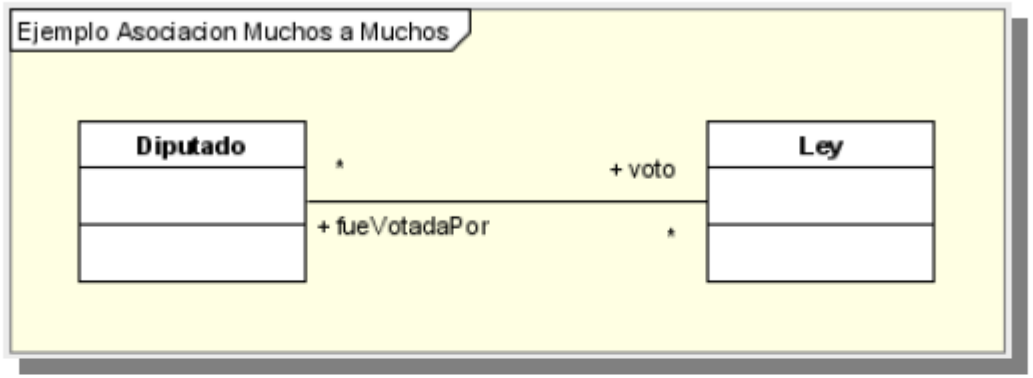
    public Persona propietario;

}
```

Decisión de Implementación

Bidireccionalidad

- Bidireccional con multiplicidad *




```
public class Diputado {

    public java.util.Collection voto = new java.util.TreeSet();

}
```

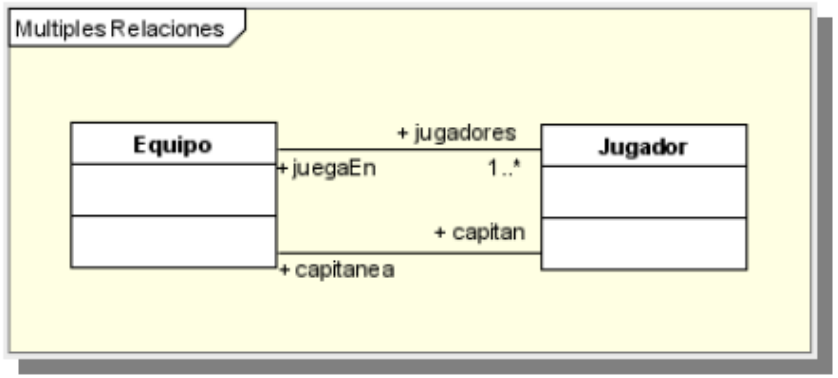
```
public class Ley {

    public java.util.Collection fueVotadaPor = new java.util.TreeSet();

}
```

Múltiples relaciones

- ¿Con más de una relación?




```java
public class Equipo {

    public Jugador capitan;

    public java.util.Collection jugadores = new java.util.TreeSet();

}
```

```java
public class Jugador {

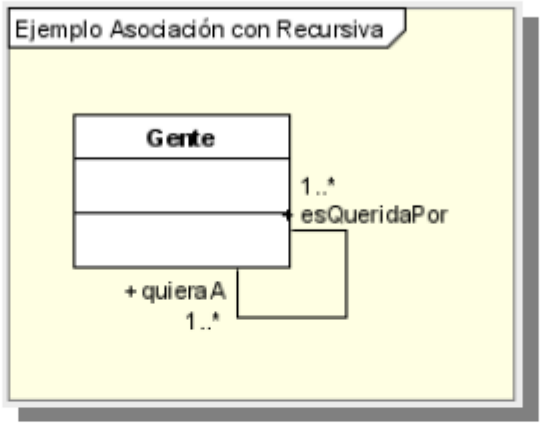
    public Equipo capitanea;

    public Equipo juegaEn;

}
```

Asociación con recursividad

- ¿Y con esto?



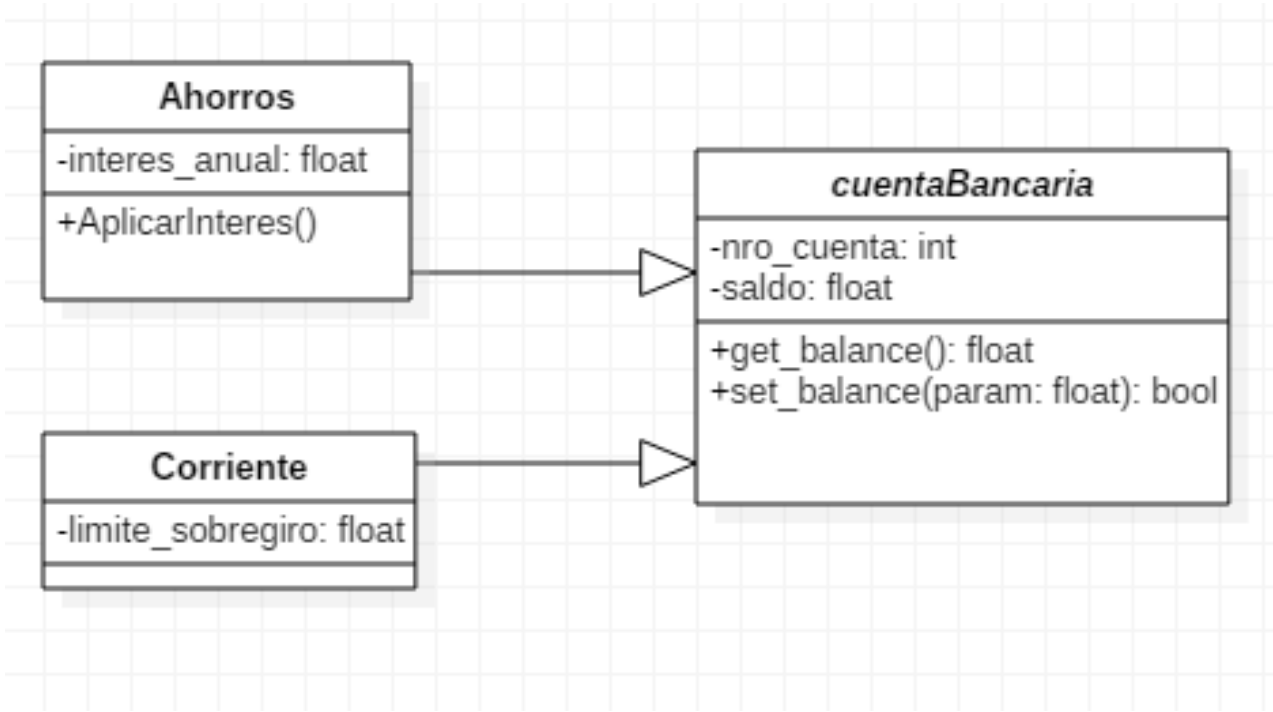

```
public class Gente {

    public java.util.Collection quieraA = new java.util.TreeSet();

    public java.util.Collection esQueridaPor = new java.util.TreeSet();

}
```

Ejemplo - Generalización



Generalización

```java
import java.util.*;

/**
 *
 */
public abstract class CuentaBancaria implements Impresion {

    /**
     * Default constructor
     */
    public CuentaBancaria() {
    }

    private int nro_cuenta;

    private float saldo;

    public float get_balance() {
        // TODO implement here
        return 0.0f;
    }

    public boolean set_balance(float param) {
        // TODO implement here
        return null;
    }

    public void ImprimirRecibo() {
        // TODO implement here
    }

    public void EstadoCuenta() {
        // TODO implement here
    }


}
```

```java
import java.util.*;

/**
 *
 */
public final class Ahorros extends CuentaBancaria {

    /**
     * Default constructor
     */
    public Ahorros() {
    }

    private float interes_anual;

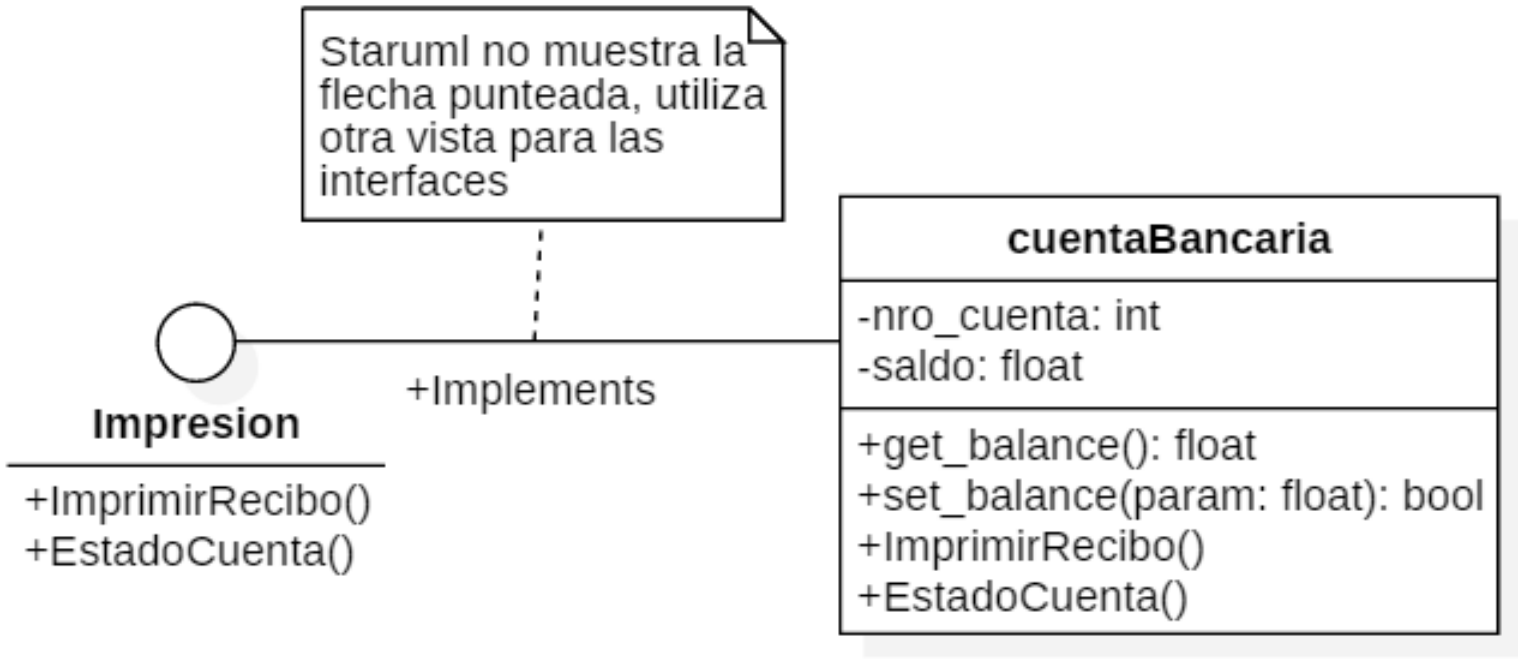
    public void AplicarInteres() {
        // TODO implement here
    }

}
```

En el DC el nombre de la clase está en cursivas
esto se traduce como clase abstracta

Ejemplo - Realización

- Impresión es una **Interface**
- CuentaBancaria **implementa** esa interface y por lo tanto desarrolla los métodos **ImprimirRecibo**() y **EstadoCuenta**()



Realización

```java
import java.util.*;

/**
 *
 */
public abstract class CuentaBancaria implements Impresion {

    /**
     * Default constructor
     */
    public CuentaBancaria() {
    }

    private int nro_cuenta;

    private float saldo;


    public float get_balance() {
        // TODO implement here
        return 0.0f;
    }

    public boolean set_balance(float param) {
        // TODO implement here
        return null;
    }

    public void ImprimirRecibo() {
        // TODO implement here
    }

    public void EstadoCuenta() {
        // TODO implement here
    }

}
```

```java
import java.util.*;

/**
 *
 */
public interface Impresion {

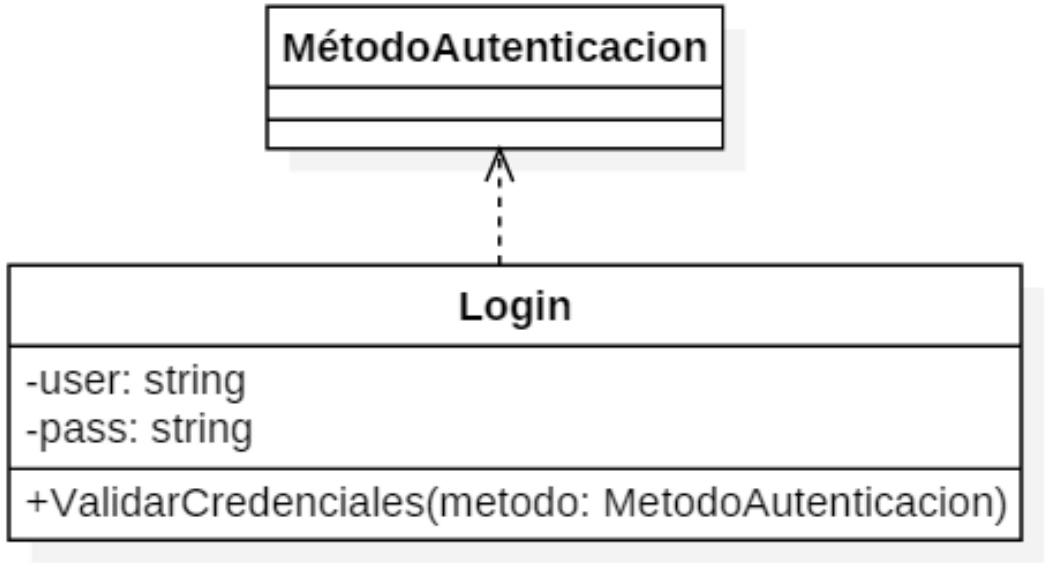
    public void ImprimirRecibo();

    public void EstadoCuenta();

}
```

Dependencia

- Indica que los cambios sobre la clase B pueden afectar a la clase A.
- ClassA usa los servicios de ClassB.
- En Java se podría representar con un import.



Dependencia

```java

import java.util.*;
import authentication.MetodoAutenticacion;
/**
 *
 */
public class Login {

    /**
     * Default constructor
     */
    public Login() {
    }

    /**
     *
     */
    private string user;

    /**
     *
     */
    private string pass;

    /**
     * @param metodo
     */
    public void ValidarCredenciales(MetodoAutenticacion metodo) {
        // TODO implement here
    }

}
```

```java

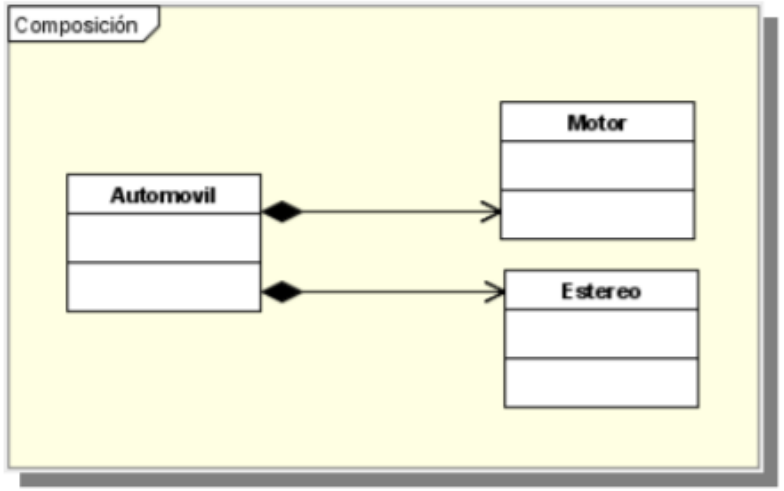
import java.util.*;

package authentication;
/**
 *
 */
public class MetodoAutenticacion {

    /**
     * Default constructor
     */
    public MetodoAutenticacion() {
    }

}
```

Ejemplificación de Composición



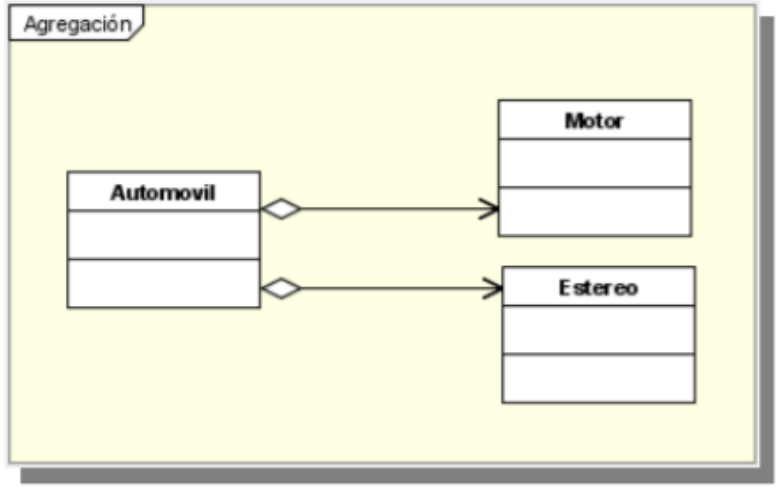

```
public class Automovil {

    public Estereo estereo;
    public Motor motor;

    public Automovil() {
        estereo = new Estereo();
        motor = new Motor();
    }

}
```

Ejemplificación de Agregación




```
public class Automovil {

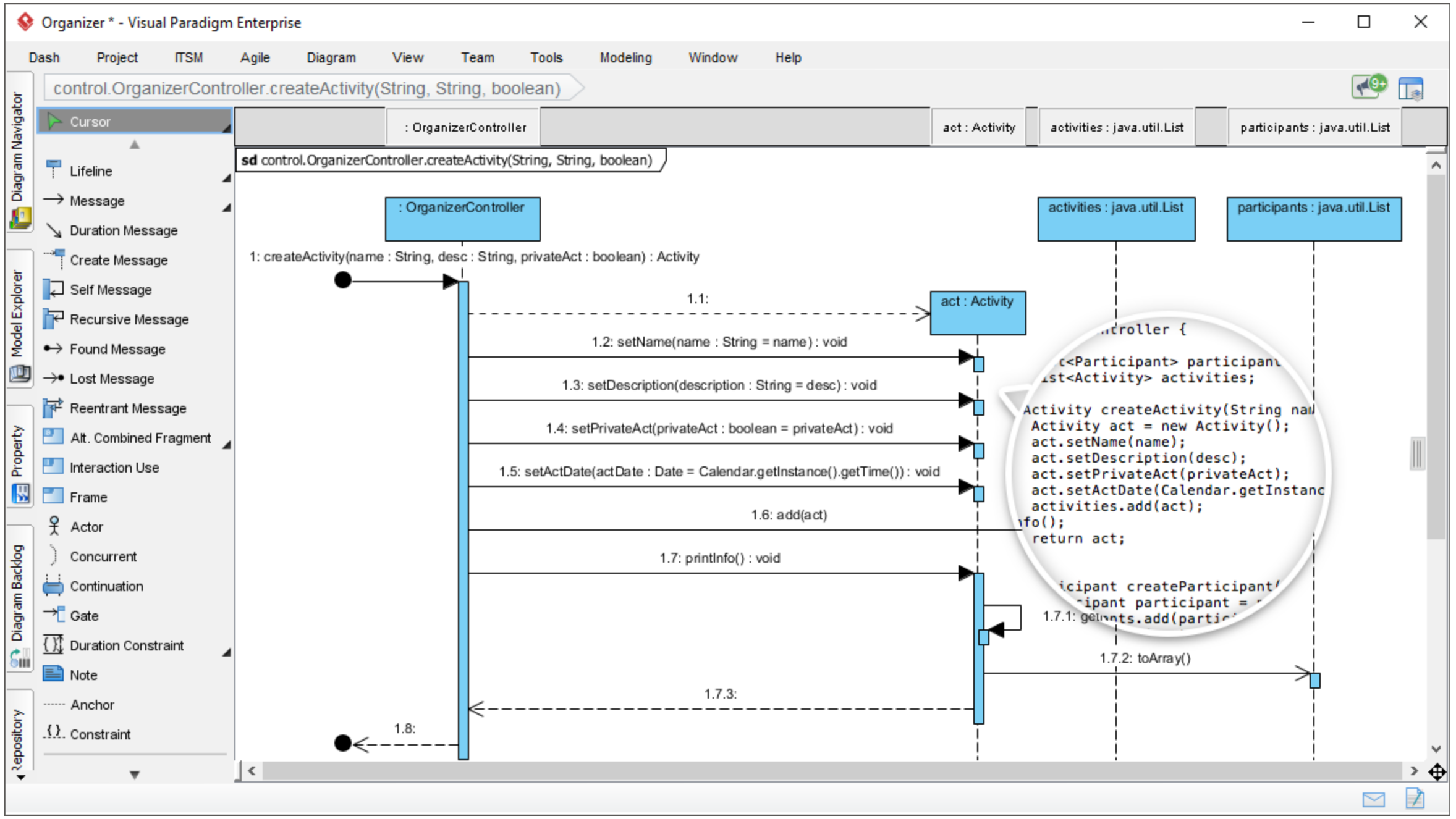
    public Estereo estereo;
    public Motor motor;

    public Automovil() {
    }

    public void ensamblar(Estereo e, Motor m) {
        estereo = e;
        motor = m;
    }

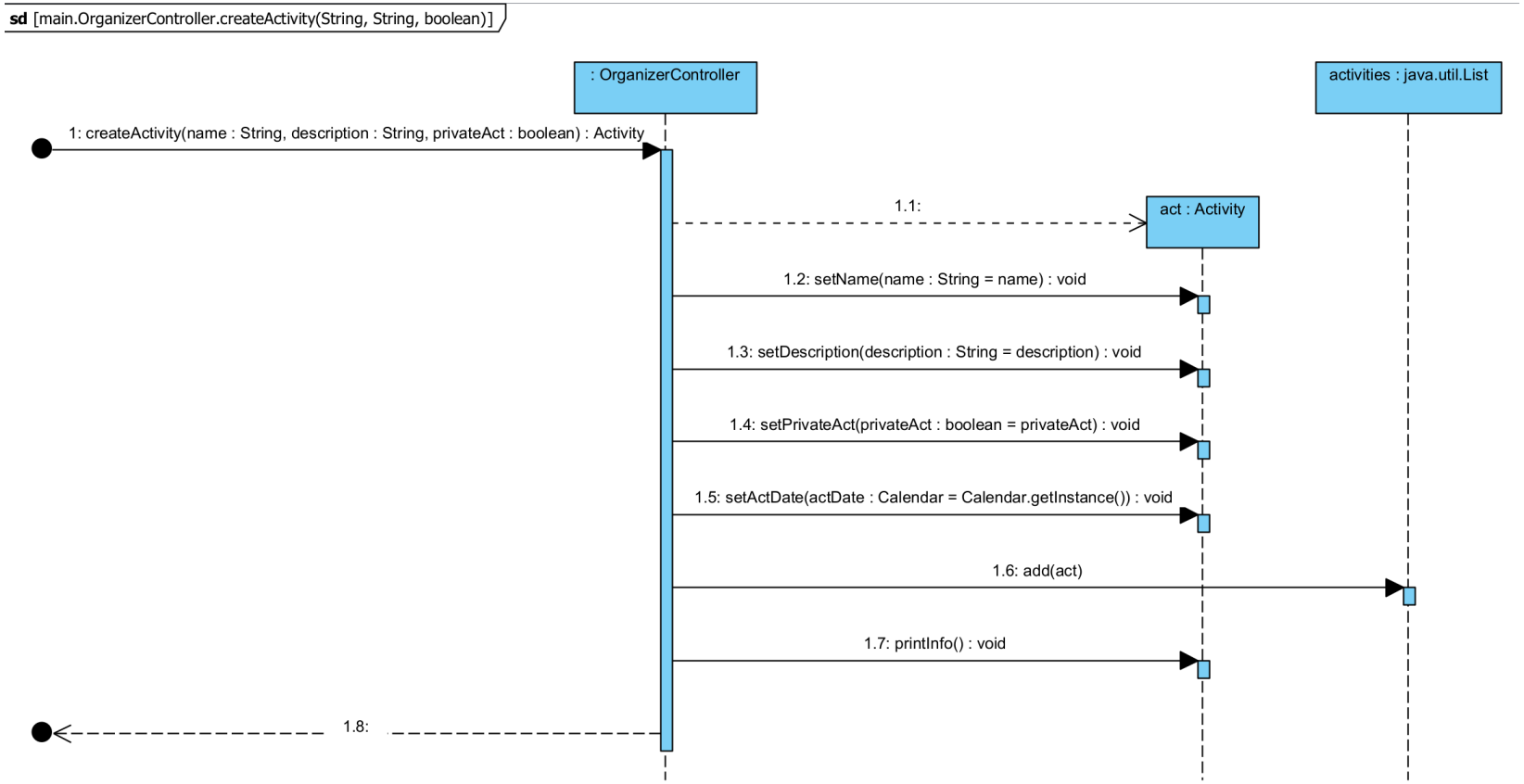
}
```

Traduciendo un diagrama de secuencia a código fuente y viceversa


Organizer * - Visual Paradigm Enterprise
Dash
Project
ITSM
Agile
Diagram
View
Team
Tools
Modeling
Window
Help
control.OrganizerController.createActivity(String, String, boolean)
Diagram Navigator
Model Explorer
Property
Diagram Backlog
Repository
Cursor
Lifeline
Message
Duration Message
Create Message
Self Message
Recursive Message
Found Message
Lost Message
Reentrant Message
Alt. Combined Fragment
Interaction Use
Frame
Actor
Concurrent
Continuation
Gate
Duration Constraint
Note
Anchor
Constraint
: OrganizerController
act : Activity
activities : java.util.List
participants : java.util.List
sd control.OrganizerController.createActivity(String, String, boolean)
1: createActivity(name : String, desc : String, privateAct : boolean) : Activity
1.1:
1.2: setName(name : String = name) : void
1.3: setDescription(description : String = desc) : void
1.4: setPrivateAct(privateAct : boolean = privateAct) : void
1.5: setActDate(actDate : Date = Calendar.getInstance().getTime()) : void
1.6: add(act)
1.7: printInfo() : void
1.7.1:
1.7.2: toArray()
1.7.3:
1.8:
Activity act = new Activity();
act.setName(name);
act.setDescription(desc);
act.setPrivateAct(privateAct);
act.setActDate(Calendar.getInstanc
activities.add(act);
return act;

De Diagrama de Secuencia a Código

- Escriba el código fuente correspondiente al método ***createActivity*** y a la clase ***OrganizerController***



De Diagrama de Secuencia a Código

```java
package main;

import java.util.ArrayList;
import java.util.Calendar;
import java.util.List;

public class OrganizerController {
    List<Participant> participants;
    List<Activity> activities;

    public OrganizerController() {
        activities = new ArrayList<Activity>();
        participants = new ArrayList<Participant>();
    }

    public Activity createActivity(String name, String description, boolean privateAct){
        Activity act = new Activity();
        act.setName(name);
        act.setDescription(description);
        act.setPrivateAct(privateAct);
        act.setActDate(Calendar.getInstance());
        activities.add(act);
        act.printInfo();
        return act;
    }
}
```

De Código a Diagrama de Secuencia

- Escriba un diagrama de secuencia para el método ***calculateTaxes***.

```java
package main;

import java.util.List;

public class Participant {
    private boolean resident;
    List<Tax> taxes;

    public Participant(boolean resident) {
        this.resident = resident;
        taxes.add(new Tax(1));
        taxes.add(new Tax(2));
        taxes.add(new Tax(3));
    }

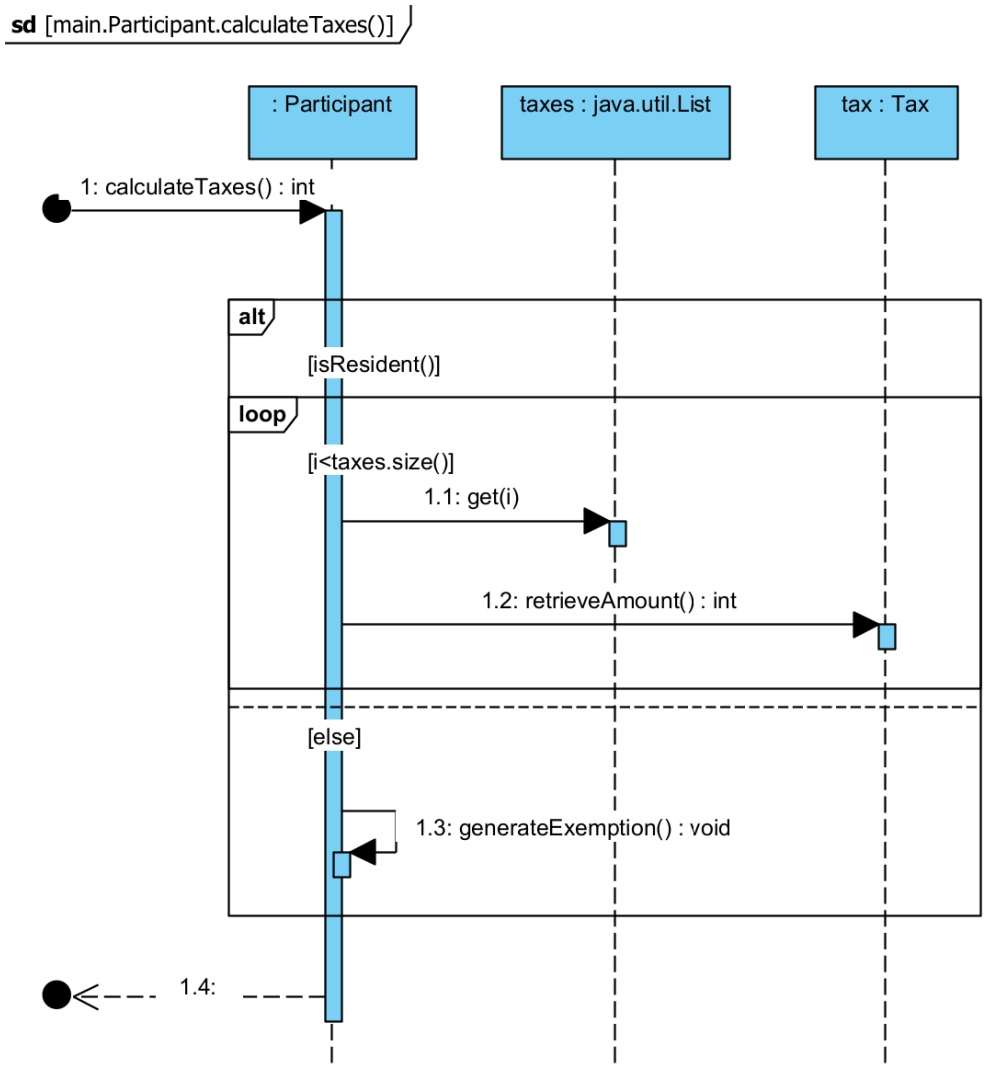
    public int calculateTaxes() {
        int total =0;
        if (isResident()) {
            int i=0;
            while (i<taxes.size()){
                Tax tax = taxes.get(i);
                total+=tax.retrieveAmount();
            }
        }
        else {
            generateExemption();
        }
        return total;
    }

    public void generateExemption() {

    }
    public boolean isResident()
    {
        return resident;
    }
}

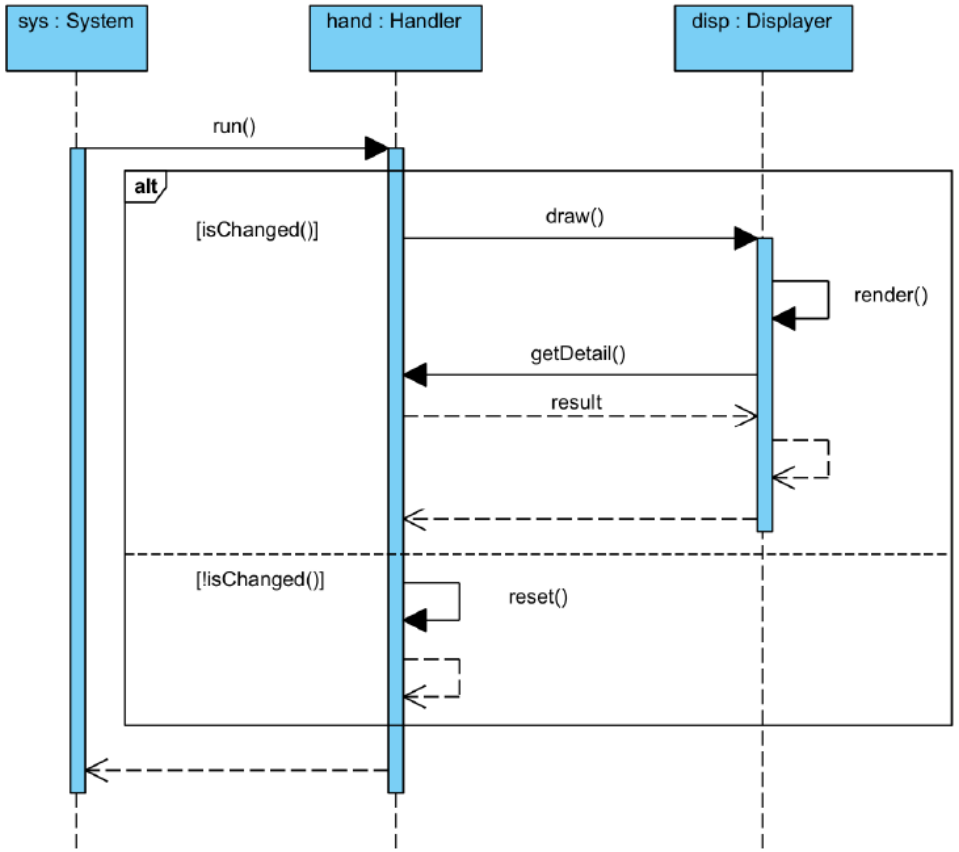
```

De Código a Diagrama de Secuencia



Ejercicio #1

Escriba el mínimo código fuente en Java para las clases *System*, *Handler* y *Displayer* que capture el comportamiento del siguiente diagrama de secuencia UML. Los cuerpos de todos los métodos deben dejarse vacíos excepto para las llamadas indicadas en el diagrama. Constructores y propiedades (es decir, métodos get y set) de las clases no necesitan ser mostradas. **[15%]**



Ejercicio #2

Escriba un diagrama de secuencia UML para la operación *calculateBill*. **[15%]**

```java
package exam2023;

import java.util.ArrayList;
import java.util.List;

public class ShoppingBasket {

    List<Item> items;

    public ShoppingBasket()
    {
        items = new ArrayList<Item>();
    }

    public double calculateBill(VAT vat)
    {
        int i=0;
        double total = 0;
        while(i<items.size()) {
            Item item = items.get(i++);
            total += item.getRawPrice();
            if (item.hasAddedValue()) {
                total += item.getRawPrice()*vat.getPercentage();
            }
        }
        return total;
    }

    public void addItem(Item item) {
        items.add(item);
    }
}
```

Ejercicio #3

Considere la siguiente descripción simplificada de una universidad donde los profesores imparten módulos en los que estudiantes pueden registrarse.

Un profesor tiene un nombre, dirección, número de teléfono, dirección de correo electrónico y salario. Un estudiante también tiene un nombre, etc., pero no tiene salario. Un estudiante, sin embargo, tiene una nota promedio (de las notas finales de sus módulos). Un módulo tiene un nombre y un número. Cuando un estudiante está registrado en un módulo, se asientan las notas de esta inscripción; y se pueden obtener tanto el promedio actual como la nota final (si la hay). De un estudiante se puede obtener la lista de módulos en los que esté inscrito. Los profesores imparten módulos. Cada módulo tiene al menos uno y como máximo tres profesores. Hay dos tipos de módulos: pregrado y postgrado. De un módulo de pregrado los estudiantes no pueden retirarse. De uno de postgrado sí.

Dibuje un **diagrama de clases** para esta universidad. Agregue atributos y métodos cuando sea necesario. No es necesario incluir *getters* y *setters* de atributos. Modificadores de visibilidad (público, privado, etc.) no son requeridos. Declare cualquier asunción que considere necesaria.

[20%]

The Law of Demeter

- It says that a method *f* of a class *C* should only call the methods of these:
  - *C*
  - An object created by *f*
  - An object passed as an argument to *f*
  - An object held in an instance variable of *C*
- If *ctxt*, *Options*, and *ScratchDir* are objects, then their internal structure should be hidden rather than exposed, and so knowledge of their innards is a clear violation of the Law.

```
final String outputDir = ctxt.getOptions().getScratchDir().getAbsolutePath();
```

- Additionally, this kind of code is often called a *train wreck* because it looks like a bunch of coupled train cars. It is usually best to split them up as follows:

```
Options opts = ctxt.getOptions();
File scratchDir = opts.getScratchDir();
final String outputDir = scratchDir.getAbsolutePath();
```

Antes de finalizar

Puntos para recordar

- Recordar cómo ir de un diagrama de clase a código fuente y viceversa.
- Recordar cómo ir de un diagrama de secuencia a código fuente y viceversa.

Lectura adicional

- Rumbauhg, Jacobson and Booch, “The Unified Modeling Language Reference Manual”
  - Chapters 1, 2, 3, 4, 5 y 8
- Perdita Stevens with Rob Pooley, “Using UML”
  - Chapters 1, 2,3, 5, 6, 7, 8 y 9
- Pressman and Maxin , “Software Engineering”
  - Appendix 1: An Introduction to UML
  - Appendix 2: Object-Oriented Concepts

Próxima sesión

- Patrones de diseño